通用人工智能创新实践课 - Agent 多智能体系统

从 ChatGPT 到自主智能体

Agent 的发展经历了从单纯的“对话系统”向“行动执行者”的演变。



2022-2023 早期

ChatGPT 崛起

带来了强大的文本生成与指令遵循能力，但只能“纸上谈兵”



2023 中期

Tool Calling 引入

工具调用 (Function Calling) 的引入，让模型拥有了“手和脚”



2023 下半年起

自主智能体爆发

Autonomous Agents 概念爆发，模型开始尝试自主规划并解决多步复杂问题

核心定义： 智能体 (Agent) = LLM (大脑) + 记忆 (Memory) + 规划 (Planning) + 工具 (Tools)

单智能体架构回顾

一个标准单智能体的核心组件可以被抽象为以下四个模块

Profile (角色设定)

定义智能体是谁，目标是什么。通常通过 System Prompt 注入。

Memory (记忆机制)

包含短期记忆（上下文窗口内的对话历史）和长期记忆（基于向量数据库的外部知识 retrieval）。

Planning (规划能力)

对复杂任务的拆解 (Task Decomposition) 与自我反思 (Self-Reflection)。如 ReAct (Reason + Act) 模式。

Action/Tools (动作与工具)

执行 API 调用、执行代码、搜索网页等对外部环境产生影响的操作。

单智能体的决策概率建模

在单智能体中，生成下一步动作 a_t 可以看作是基于当前状态 s_t （包含记忆、环境观察与角色）的条件概率分布：

$$P(a_t|s_t, h_{<t}) = LLM\ (a_t|\text{Prompt}(s_t, h_{<t}))$$

注： $h_{<t}$ 代表历史交互轨迹 (Trajectory) 。单智能体的性能上限受限于 LLM 对这一复杂条件概率分布的拟合能力，以及上下文窗口的长度限制。

单智能体的局限性

虽然单智能体在简单任务上表现出色，但在处理工业级复杂项目时，面临着诸多瓶颈。

1、幻觉的累积效应 (Hallucination Cascade)

- 只要在多步推理中的某一步产生幻觉，后续的所有规划都将建立在错误的前提之上。
- 缺乏外部视角的交叉验证。

2、上下文窗口灾难 (Context Explosion)

- 将复杂的代码库、长篇文档和历史日志全部塞入单一窗口，会导致 LLM 的“中间遗忘” (Lost in the Middle) 现象。

3、单一角色的能力瓶颈

- 要求一个单一的 Prompt 兼顾“架构师”、“程序员”和“测试员”的视角，往往会导致指令冲突，模型陷入混乱。

4. 模型同质化与成本妥协 (Model Heterogeneity)

- 单智能体只能调用单一模型。但在工程实践中，我们往往需要让“推理能力强、昂贵的大模型”担任架构师，而让“速度快、便宜的小模型”处理基础解析，以此来平衡效果与成本。

异构 LLM (Model Heterogeneity)

不同模型因数据来源、算法架构、训练侧重点的不同，展现出了截然不同的“天赋”，多智能体架构允许我们集百家之长。

</> 深度推理与后端架构

代表： Claude 5 Opus, GPT-5, Kimi k3, 智谱 GLM-5

擅长全局架构设计、复杂逻辑推理和核心算法编写。

文案创作与知识整合

代表： DeepSeek, Gemini 3.1 Pro, 文心 5.0

词汇丰富、表达自然，擅长长篇写作、文献综述和逻辑梳理。

前端审美与多模态处理

代表： Gemini 3.1 Pro, GPT-5o

拥有出色的前端 UI Taste，同时原生支持端到端的音视频/图像解析。

极速任务与预处理

代表： Claude 5 Haiku, Gemini 3.7 Flash

极低延迟、极低成本。适合做海量日志清洗、API路由和网页抓取。

💡 **最佳实践是为不同任务分配最适合的大模型！**

多智能体系统优势（一）：任务拆解与专业化分工

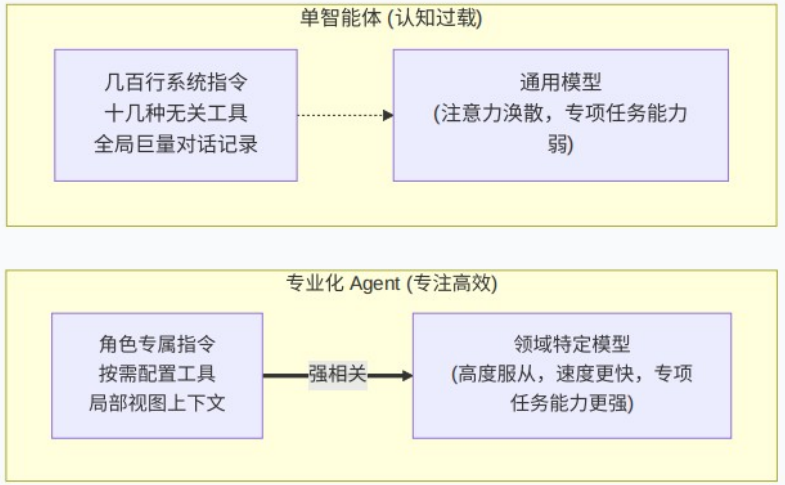
单智能体处理复杂业务时，极易因“上下文过载”而崩溃。多智能体系统通过将巨型目标拆解为独立子任务，交由专业 Agent 协作完成。

专业化分工的核心优势：

🎯 **降低信息干扰，聚焦注意力**：通过设定独立的系统指令，仅传递上下文的**局部视图** (Local View)，且只配备特定工具。这避免了 LLM 因无关信息干扰而导致注意力分散，极大保证了推理深度和指令服从能力。

📏 **平缓上下文增长与扩展边界**：相比单智能体将所有信息塞入单一窗口，多智能体隔离了状态流转，大幅降低了单次交互的上下文需求。在相同的上下文窗口 token 长度限制下，能处理规模更大的复杂任务。

🛠️ **极佳的局部调优与迭代能力**：如果某个环节表现不佳或需引入新能力，开发者只需修改、替换或升级该特定 Agent （如给“数据分析 Agent” 单独更换更强模型或增加工具），无需对全局系统进行重构。



多智能体系统优势（二）：容错机制与交叉验证

在大语言模型的推理过程中，存在一个不对称性：**验证一个解答的正确性通常比凭空生成该解答更容易。** MAS 利用此特性，能大幅降低推理结果遗漏任务要求的概率。

典型验证拓扑：生成器-鉴别器模式 (Generator-Critic)

这与 GAN (生成对抗网络) 的思想异曲同工。由 **Work Agent** 负责生成代码或推理草案，由具备不同系统提示词（甚至是不同基座模型）的 **Evaluator Agent** 负责边界审查或用例测试。这种基于多视角的制衡机制，打破了单一模型自我纠错 (Self-Correction) 能力薄弱的系统局限。

多智能体辩论 (Multi-Agent Debate)

让多个 Agent 对同一复杂问题给出独立解答，并在互相审查和辩论中修正逻辑谬误，最终达成共识。学术界已证明该机制能显著提升数学和复杂逻辑数据集的准确率。

多专家投票与集成 (Ensemble Voting)

借鉴机器学习的集成思想，针对高风险决策，可并发调用多个配置不同角色（或不同异构基座）的 Agent 进行独立推断。通过多数表决 (Majority Voting) 或加权评分机制，利用群体智慧最大概率过滤单一模型的偶发性幻觉。

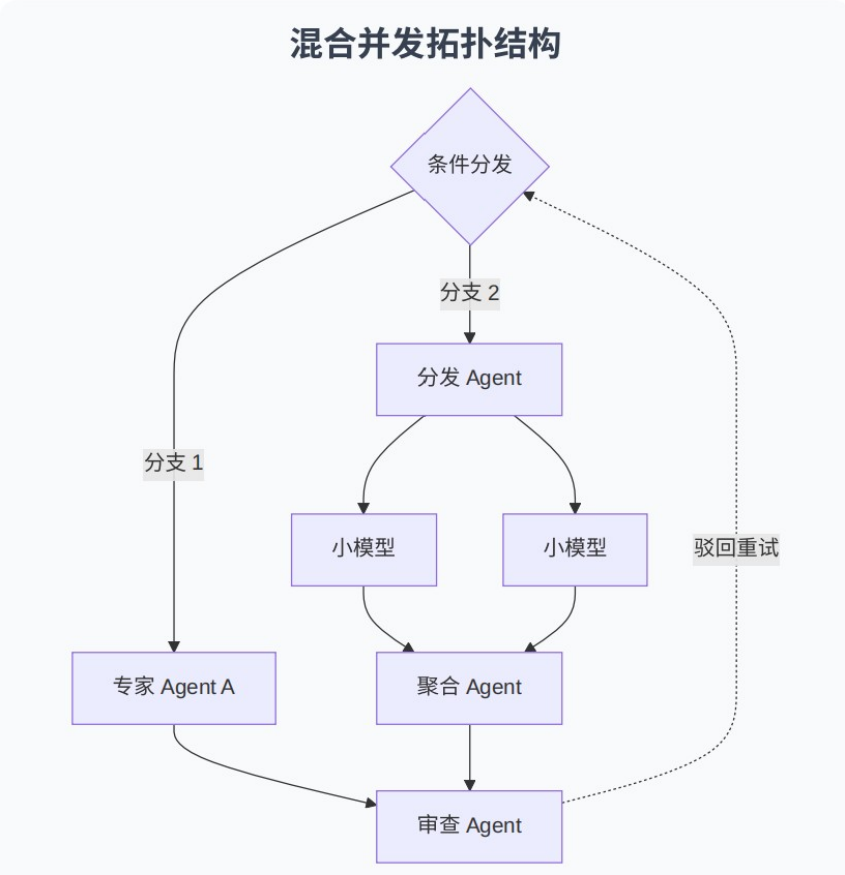
多智能体系统优势（三）： Workflow 编排与受控并发

随着复杂性的提升，纯线性的推理链已无法胜任。 MAS 引入了 Workflow (工作流) ，实现了工业级的控制流管
理与效率飞跃。

流程控制与路由 **(Control Flow)**： 通过 workflow ，将复杂业务逻辑（如循环、条件分支）转化为节点连线。这种结构实现了确定性的路由流转，使执行流高度可控且可视化。

面向 **Agent** 的 **Map-Reduce** 范式： 面对海量任务， Dispatcher Agent 会将任务派发给底层 “小模型 Agent” 并行处理，最后由 Aggregator 汇总，极大缩短系统 Latency 。

摒弃纯异步，拥抱状态机： 早期框架（如 AutoGen ）尝试纯异步消息传递，却暴露出追踪难、 Debug 灾难等痛点。目前业界普遍采用 Workflow 进行受控并发，既保证了并行效率，又实现了强回溯与可中断恢复能力。



什么是多智能体系统（MAS)?

基于 LLM 的多智能体系统 (Multi-Agent System, MAS) 继承了分布式系统的思想，但其计算节点变成了具备自然语言理解和推理能力的 LLM Agents 。

传统 **MAS**

- 基于固定规则和逻辑门
- 节点间传递高度结构化的数据包
- 协调机制合同机制、博弈论
- 可靠性，有形式化保证，但环境受限
- 强耦合，缺乏鲁棒性应对未见场景

LLM **MAS**

- 基于自然语言和语义推理
- 节点间传递模糊的意图和自然语言 (Prompt)
- 角色分工、对话协商、涌现协作
- 灵活通用，但易出现幻觉、死循环、不可控
- 泛化能力强，通过交互涌现出解决问题的策略

核心通信拓扑 1：串行工作流（Sequential）

最基础且最稳定的模式，各个 Agent 按照预设的顺序依次接力执行任务。



- **工作机制**：前一个 Agent 的输出，直接作为下一个 Agent 的输入上下文。
- **适用场景**： SOP（标准作业程序）明确、步骤线性的任务。例如：内容创作流程（资料搜集 -> 大纲 -> 撰写 -> 润色）。
- **优点**：极度稳定，易于监控每一步的产出， Token 消耗可控。
- **缺点**：缺乏灵活性。如果前面的环节出错，错误会一直向后传递，且难以回头重试

核心通信拓扑 2：层级式 (Hierarchical)

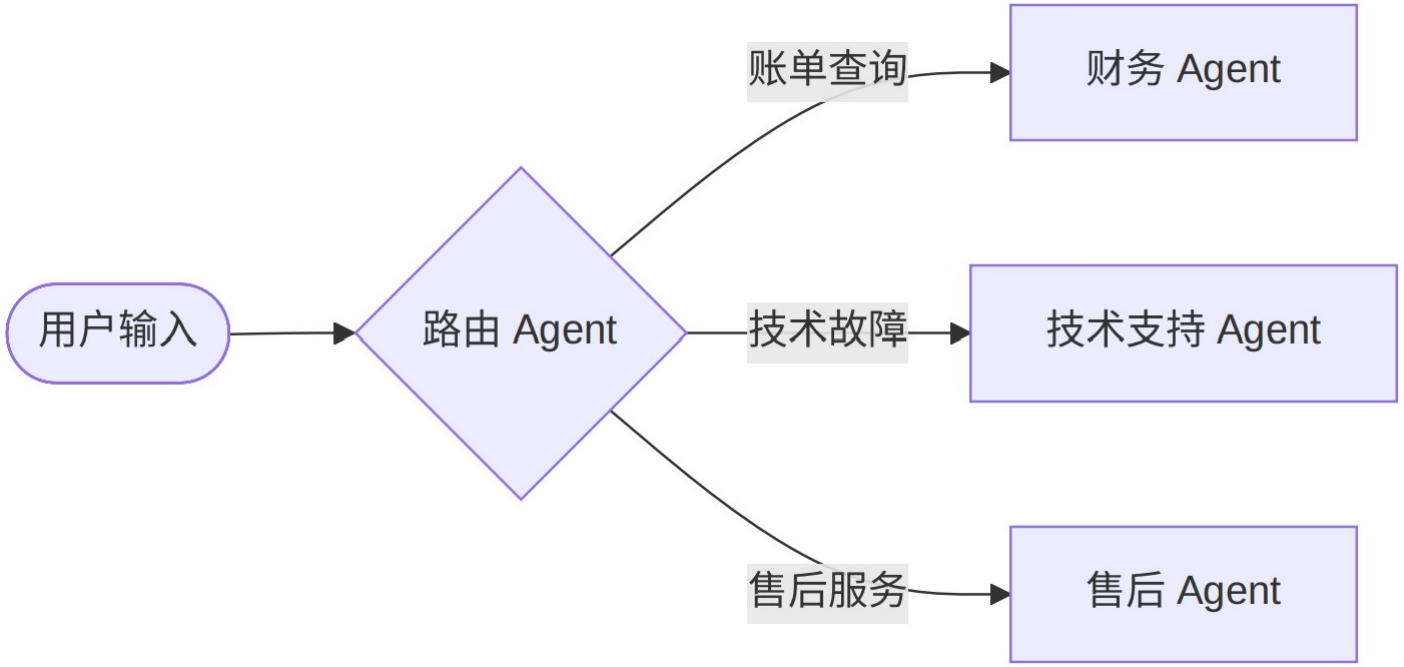
模拟人类公司的组织架构，由一个 Manager Agent 负责任务拆解和统筹，多个 Worker Agent 负责具体执行



- **适用场景：**大型复杂项目的自动生成（如完整的软件开发流程），需要多工种协作。
- **优点：**分工明确，扩展性好，可以轻松增加新的工种 (Worker)。
- **缺点：** Manager Agent 容易成为性能瓶颈，其上下文窗口可能因汇集所有 Worker 的汇报而溢出。

核心通信拓扑 3：中心路由式（Supervisor/Router）

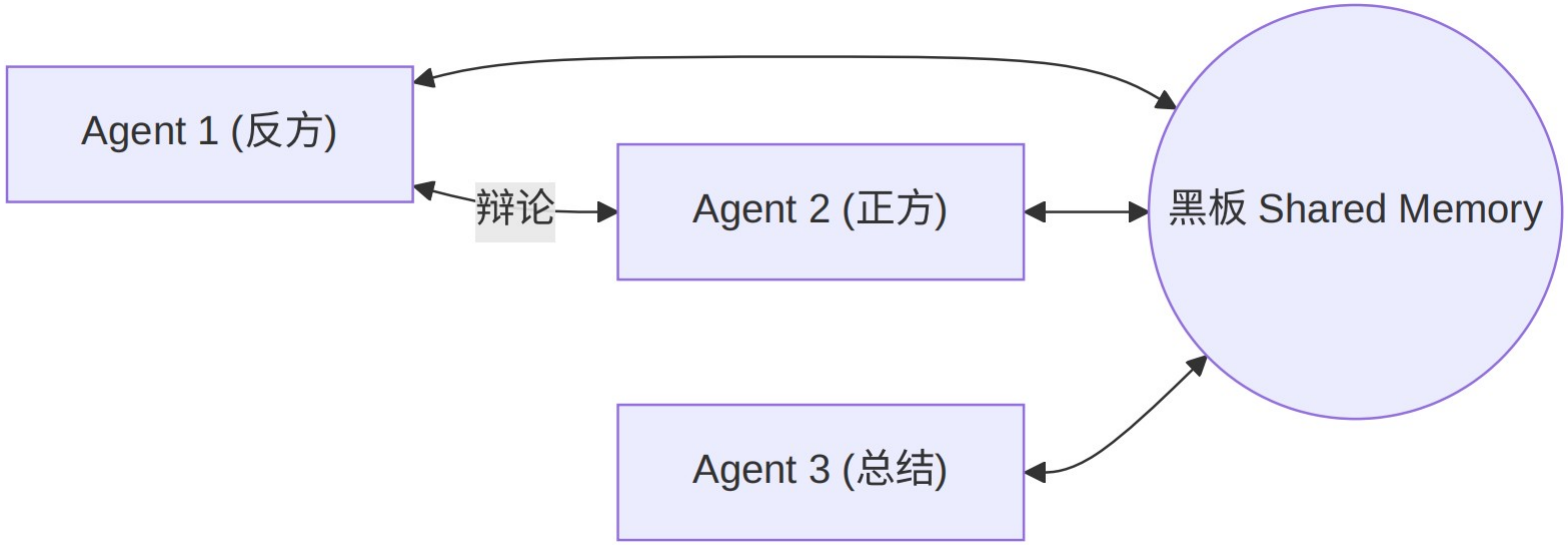
基于意图识别的星型拓扑，由 Router 决定当前任务该交给谁处理。



- **适用场景**：用户意图发散的综合入口（如企业全能客服系统、系统级 Copilot ）。
- **优点**：对外接口统一，模块解耦。
- **缺点**：强烈依赖 Router 的意图识别能力。如果路由错误，后续处理全盘皆输

核心通信拓扑 4：去中心化 / 黑板机制（P2P/Blackboard）

所有 Agent 监听一个公共的“黑板”（共享内存）或进行点对点的自由辩论，基于上下文自主决定是否介入。



适用场景：开放式创新、复杂逻辑推理、头脑风暴。

优点：能够激发出 LLM 最强的“涌现能力 (Emergent Abilities)”，通过不同视角的碰撞消除局部幻觉。

缺点：极难控制，极易陷入死循环 (Infinite Loops)，消耗天量的 Token，在工程上落地最困难。

A2A 协议（Agent-to-Agent Protocol）

A2A 协议是用于 AI 智能体之间通信的开放标准。它为不同框架和供应商的智能体提供了一种通用的交互语言，打破了生态孤岛，使得跨组织协作成为可能。

⚠ 传统的集成挑战

- **Agent 暴露受限**：通常将智能体包装为普通工具，限制了其自主协商能力。
- **定制化集成**：每次交互都需要定制化点对点开发，工程开销巨大。
- **扩展与安全瓶颈**：随着智能体数量增加，系统难以维护，且缺乏一致的安全措施。

✓ A2A 带来的优势

- **标准化交互**：提供原生的标准化协作方法，无需复杂的定制开发。
- **无须工具包装**：直接暴露智能体本体，保留其自治性和复杂决策能力。
- **加速创新生态**：不同框架的 Agent (如机票预订、酒店预订) 能够无缝协作。

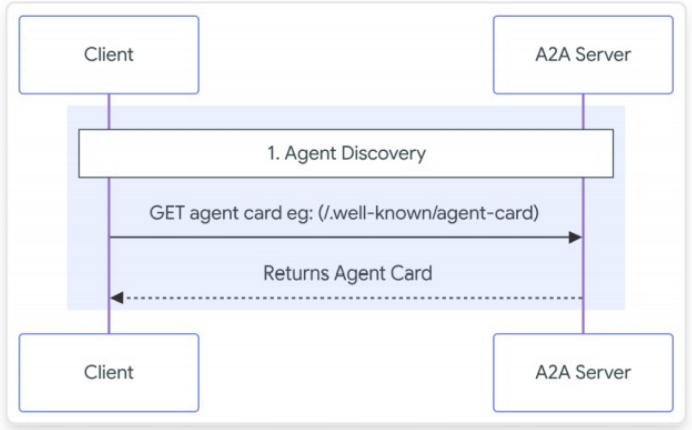
A2A Protocol: What is A2A

A2A 请求生命周期（Request Lifecycle）

一次完整的 A2A 通信包含三个主要阶段。客户端、 A2A 服务端以及认证服务器的交互如下：

1. 智能体发现 (Discovery)

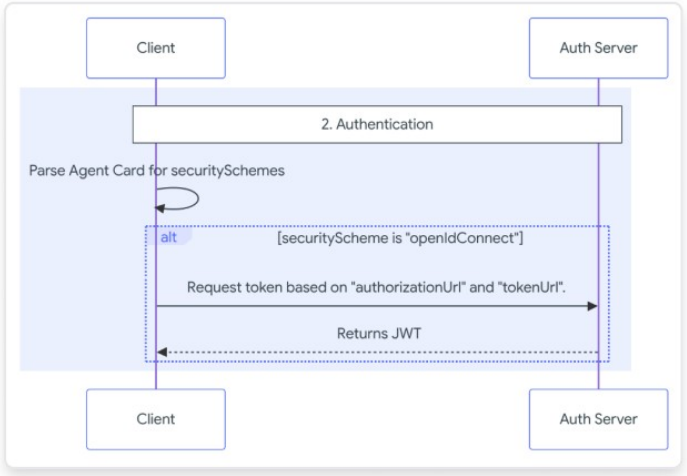
客户端获取服务端的 `Agent Card`，了解该智能体的能力和端点信息。




```
{
  "name": "Research Agent",
  "description": "Finds and summarizes primary sources.",
  "version": "1.0.0",
  "protocolVersion": "1.0",
  "supportedInterfaces": [
    { ... }
  ],
  "provider": { ... },
  "capabilities": { ... },
  "defaultInputModes": ["text"],
  "defaultOutputModes": ["text"],
  "skills": [ ... ],
  "securitySchemes": {
    "oauth2": { ... }
  },
  "security": [{ "oauth2": ["read"] }]
}
```

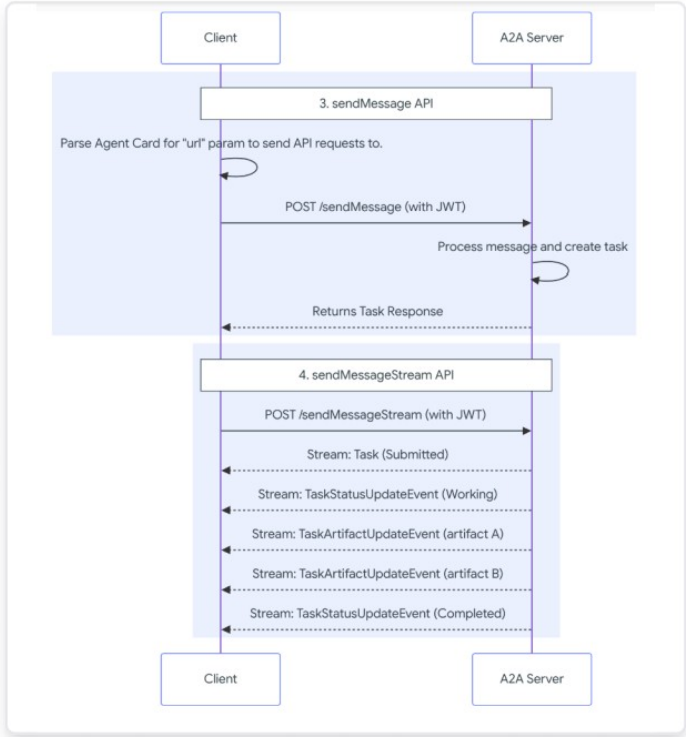

2. 身份认证 (Authentication)

客户端读取安全方案。在需要时，向 Auth Server 请求并获取 Token。



3. 消息交互 (Messaging)

通过 `sendMessage` 或 `sendMessageStream` 向服务端发送消息以执行任务。



A2A vs MCP

在现代的 Multi-Agent 系统中，通信被严格分层：

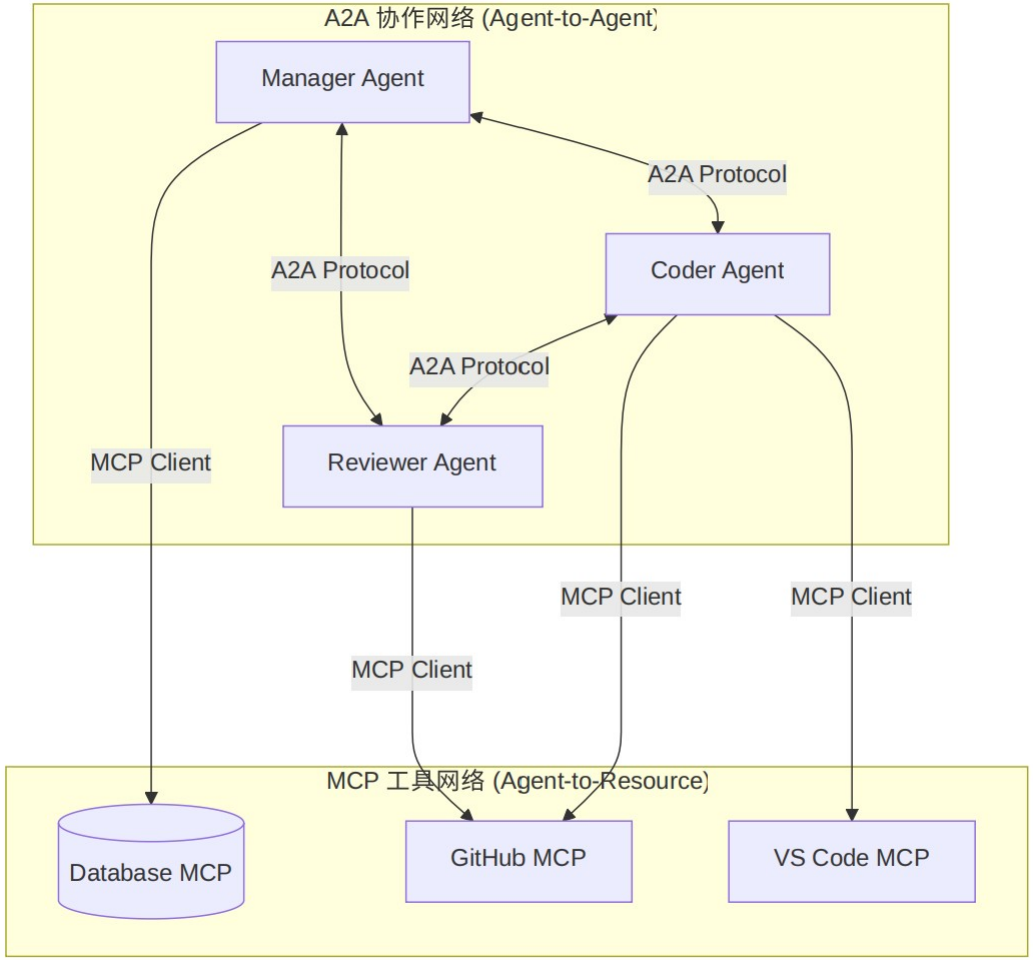
特性	A2A (Agent-to-Agent)	MCP (Model Context Protocol)
主要目标	智能体之间的任务委托、身份发现与状态同步	智能体与外部工具、本地资源、数据的连接
层级	协作层 (Orchestration Layer)	基础设施层 (Infrastructure Layer)
核心机制	Agent Cards, Delegation, Msg Passing	Server-Client 模型, 工具描述注入
解决的痛点	不同框架 (如 LangGraph 调 CrewAI) 间的互操作	避免为每个模型手写 API 适配器

💡 **典型的调用链路：**

用户触发 ➡ 主管 Agent 通过 [A2A 协议] 委托任务给 数据分析 Agent　　数据分析 Agent 通过 [MCP 协议] 获取数据

A2A 与 MCP

A2A 与 MCP 是协作关系，并不冲突



Agent Network Protocol (ANP) 架构

ANP 1.1 专为开放、跨域的智能体网络设计，采用优雅的三层架构

- **Application Protocol Layer**（应用层）包含智能体

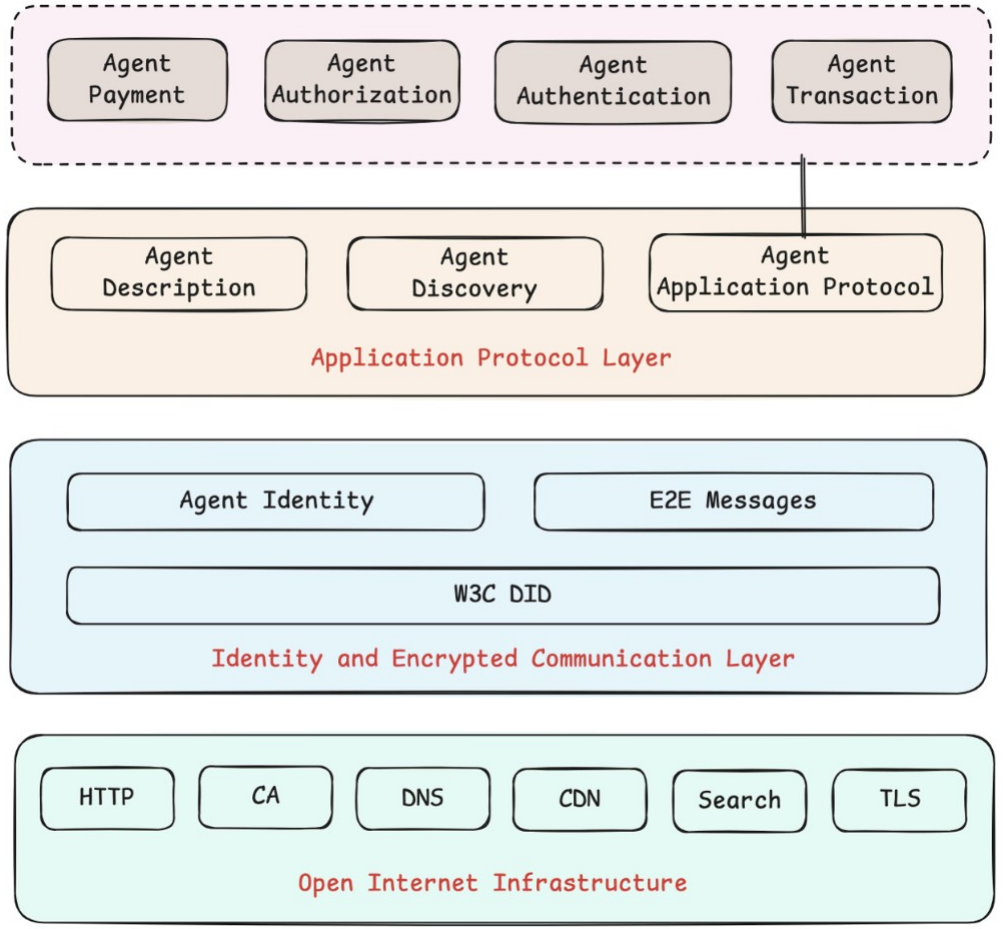
描述 (Agent Description)、发现 (Agent Discovery)，以及特定领域的协议（如支付、授权、认证、商业等）。

- **Identity & Communication**（身份与通信层）提供

DID: WBA 身份、 WNS 标识、认证、密钥分发、安全消息以及跨域通信的基础设施。

- **Open Internet Infrastructure**（开放网络基础设施）在底层复用现有的 HTTP, CA, DNS, CDN, Search, TLS 以及 Web 的成熟运维模式。

官方文档
: Agent Network Protocol



ANP 协议栈与工作流

从底层身份到上层应用， ANP 提供了完整的协议栈支持

- **基础设施复用（Open Infrastructure)**：依托 HTTP,

DNS,TLS 等开放基建。

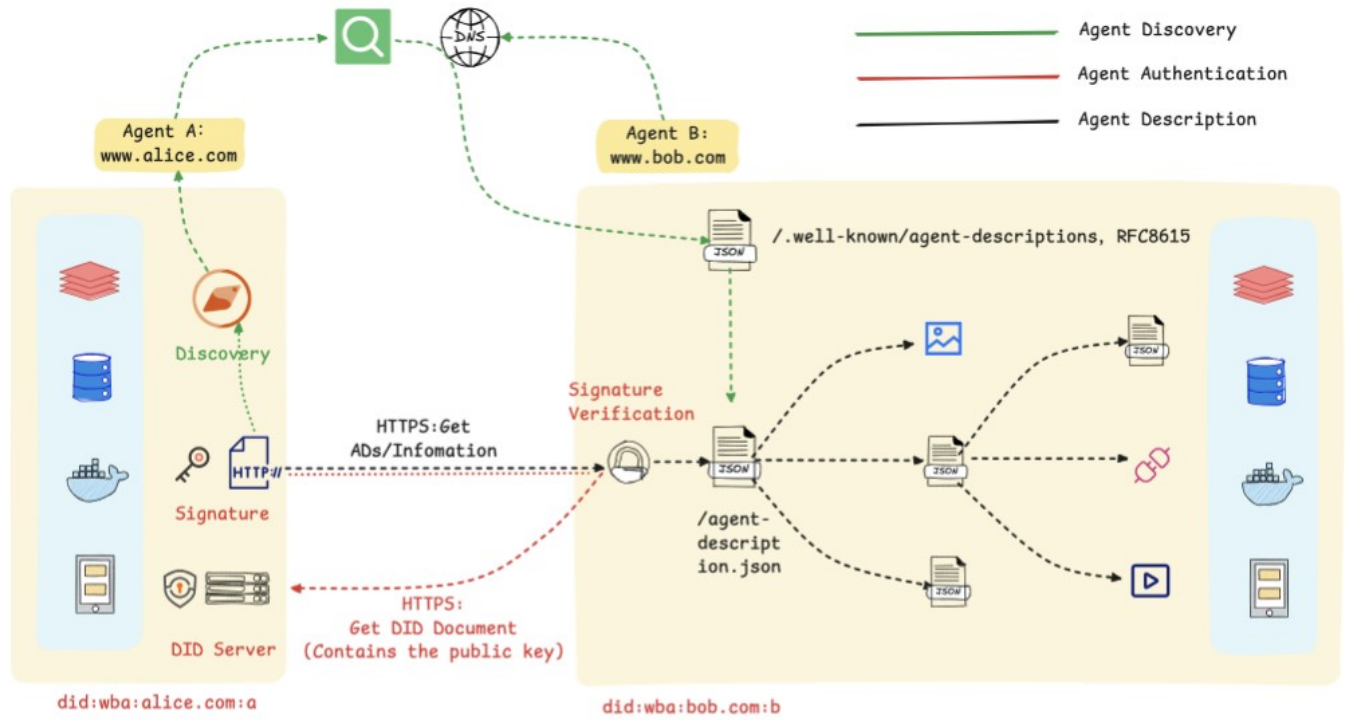
- **身份与通信（Identity & Comm)**：DID:WBA,WNS 和

端到端加密（E2EE)，构建跨域基础。

- **描述与发现（Description & Discovery)**：智能体通过标准化 JSON 描述自身能力和端点，供其他智能体调用。
- **消息配置（Messaging Profiles)**：提供直连、群组、

附件及联邦通信等专注的收发方案。

- **领域协议（Domain Protocols)**：顶层提供支付、商业等垂直场景协议。



MAS 主流框架对比：LangGraph vs CrewAI

维度	LangGraph	CrewAI
核心理念	Graph-based State Machine (基于图的状态机)	Role-based Teams (基于角色的团队)
控制力	极高 (可精准控制每一个边和节点的流转)	中等 (框架封装了大量底层协调逻辑)
状态管理	强 (Checkpointers 支持持久化与时间回溯)	弱 (以任务驱动，状态对开发者较黑盒)
开发心智	工程师导向 (定义图、节点、边)	产品导向 (定义谁来做，做什么)
适用场景	企业级复杂、需人工干预的重型工作流	快速原型的轻量级内容生成、分析团队

选择建议： 如果系统失败成本极高，需要精准控制和断点调试，选 **LangGraph**。如果想让几个 Agent 合作写一篇文章，选 **CrewAI**。

代码范式对比（1/2): LangGraph（定义节点与边）

在 LangGraph 中，一切皆是“图”。开发者必须明确定义状态 (State) 以及状态流转的路径。

```python
# LangGraph: 关注图的拓扑与状态流转
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]

def research_node(state: AgentState):
    response = model.invoke(state["messages"])
    # 显式更新状态
    return {"messages": [response]}

workflow = StateGraph(AgentState)
workflow.add_node("researcher", research_node)
workflow.add_node("writer", writer_node)
workflow.add_edge("researcher", "writer") # 明确流转路径

app = workflow.compile()
```

代码范式对比（2/2): CrewAI（定义角色与任务）

在 CrewAI 中，开发者更像是一个 HR 和项目经理，只需定义角色和任务，由框架处理底层的通信。

```
# CrewAI: 关注角色和职责
researcher = Agent(
    role='Senior Research Analyst',
    goal='Uncover cutting-edge developments in AI',
    backstory='You are a seasoned analyst at a top tech think tank.'
)

task1 = Task(
    description='Analyze 2026 AI Agent trends',
    expected_output='A bulleted list of 5 key trends',
    agent=researcher
)

crew = Crew(
    agents=[researcher, writer],
    tasks=[task1, task2],
    process=Process.sequential # 仅需声明协作模式
)

result = crew.kickoff()
```

框架的演进：走向融合（Convergence）

随着技术的发展和时间的推移，不同智能体框架不断相互吸收优点，它们的差异和设计理念**已经不再泾渭分明**。

早期界限分明

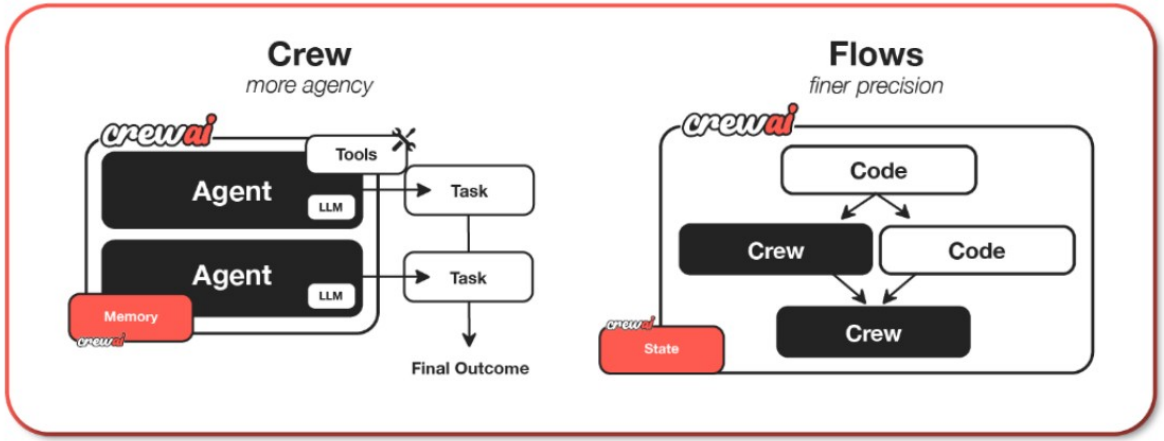
LangGraph 追求极致的底层流转控制；
CrewAI 侧重高度抽象的角色分配。

现在的融合趋势

CrewAI 引入 Flows 弥补流程控制；
LangGraph 向上封装 降低上手门槛。

未来的终局

底层基于统一的通信与工具协议，
上层提供既灵活又易用的编排模式。



CrewAI 官方架构图：从纯粹的 Crew（高度自主）到 Flows（细粒度控制），吸收了类似 LangGraph 的状态机理念

现实设计中的核心挑战（The "Dark Side" of MAS）

在实验室写 Demo 很容易，但在生产环境中部署多智能体，往往面临以下难题

1. **上下文爆炸与同步（Context Explosion）**
多个 Agent 不断交流，对话历史成指数级增长，不仅导致高昂的 API 成本，还会快速耗尽模型的 Token 上
限，引发“遗忘”。

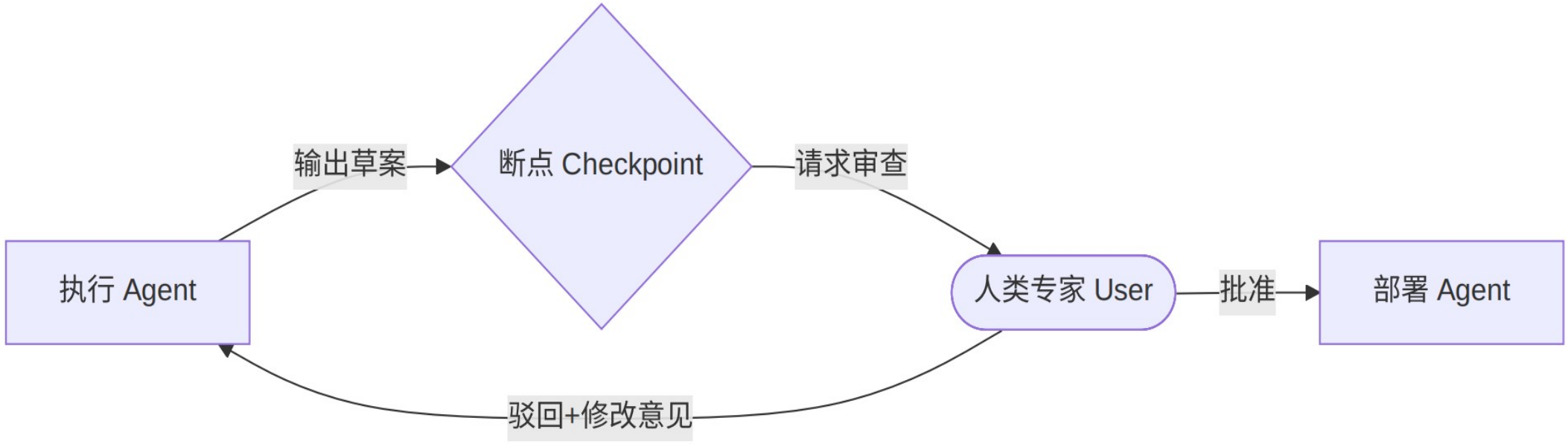
2. **状态发散与死循环（Infinite Loops & Hallucination Cascades）**
没有强护栏（Guardrails）的自由交流（如 P2P 拓扑），经常会导致 Agent 相互恭维，或在错误的前提下反复兜圈子。

3. **延迟（Latency）**
多步 Agent 推理是串行的阻塞调用，一个任务可能需要几十秒甚至几分钟才能返回。无法用于 C 端高频实
时交互场景。

4. **多智能体共识机制（Consensus）**
当“安全审查 Agent”和“代码生成 Agent”意见不合时，谁来拍板？（需引入投票机制或 Human-in-the_x0002_
loop 人工干预）。

应对挑战：人机协同（Human-in-the-loop, HITL）

不追求 100% 自动化，而是将“人类”作为图谱中的一个节点。



Time Travel（时间回溯）：像 LangGraph 这样的框架支持持久化 Checkpoint。如果后面的一步出错了，可以手动回滚到上一个节点，修改 Agent 的状态后重新执行。

工业界案例剖析： Coding Agents 的架构取舍

以当前最顶尖的 AI 编程智能体为例，看看工业界是如何在架构上做取舍的。

Cursor

并发多智能体探索

通过 Agent Window 并发执行多个策略。

架构取舍

牺牲单个任务思考深度换取并行速度；始终保持开发者在环 (HITL)，适合高频小步快跑。

Claude Code

Plan-Execute 子智能体

由主 Agent 根据需要派生子 Agent 执行。

架构取舍

主从单向通信，且子Agent通常只读，只允许主Agent负责写操作；避免并发修改相同模块或文件引发剧烈代码冲突。

Devin

全自动异步沙盒

运行在云端隔离环境，自我反思纠错。

架构取舍

产生极高时间延迟，完全牺牲实时交互性；换取解决 Issue 的高完备率和 Fire-and-forget 体验。

如何评估多智能体系统（Evaluation)?

目前学术界和工业界一般采用如下评估体系

端到端任务成功率（End-to-End Success Rate)

- 最终目标是否达成？（例如：代码是否通过了所有 Unit Tests，测试覆盖率是否达标）。SWE-bench 是典型代表。
- 幻觉：多智能体系统是否有效降低了幻觉频率?

推理流程 (Trajectory)

Agent 之间是否发生了太多的冗余推理和冲突?

系统级指标（System Metrics)

Token 效率：更好的结果，是否只是因为创建了更多的智能体花费了更多的 token，思考了更久?

耗时：多智能体系统是否通过并发真正实现了任务提速

总结与未来趋势

本课程核心回顾：

1. 单智能体无法应对工业级复杂系统的多视角与长流程需求。
2. 拓扑结构决定了系统的上限与稳定性，从串行到网状架构，难度递增。
3. **MCP** 解决了工具连接， **A2A** 协议解决了智能体协
作，进入标准化时代。
4. 工业界（如 Cursor, Claude Code ）在延迟、自主性
与并发间做出了不同的工程取舍。

未来趋势预测：

1. 多模态原生协作： Agent 之间不仅传递文字，直接传递视觉和空间张量图。
2. 端侧智能体网络 (On-device MAS)：个人手机、PC 与云端巨模型形成分布式协作网，解决隐私计算问题。
3. 自我进化：通过多智能体在沙盒中无休止的对抗 (Self-Play) ，产生于微调下一代模型的高质量合成数据。

实战任务：将搜索工具升级为“子智能体”

🛑 问题：重型Tool（如搜索）的瓶颈

当主 Agent 直接调用“网络搜索”等会返回巨量文本的工具时：

- 一次性向上下文填入数十上百 K 的 Token。
- 只需几次调用就会触及LLM输入长度上限，且推理成本极高。
- 主 Agent 的核心推理能力被冗长的噪音信息稀释。

🎯 任务：引入 Search SubAgent

请尝试为你现有的单 Agent 系统添加**子智能体机制**：

- 将直接暴露给主 Agent 的网络搜索 Tool，**替换**成一个专注于搜索和提炼的 `Search SubAgent`。
- 主 Agent 遇到需要检索的问题时，调用 SubAgent，只获取提炼后的高价值总结。





架构设计：开放性工程思考题

引入子智能体在框架设计上会面临诸多取舍和挑战，请在动手前思考以下核心问题

1. 服务发现与触发机制

主 Agent 怎么发现子 Agent 的存在，并知道何时调用它？是将它**注入到系统指令中**，还是通过**skill披露**？

2. 调用协议与 Schema 设计

接口应该是什么样？模型 ID、System Prompt、Tools 等，哪些允许**动态指定**？哪些必须被**框架写死**？

3. 生命周期与交互模式

子 Agent 是任务完成后立即销毁（`Stateless`），还是保持会话状态，允许主 Agent 进行**多轮追问**？

4. 并发与阻塞调度

派发任务后，是否应该**阻塞 (Block)**主 Agent？如果非阻塞，主 Agent 在等待期间如何挂起或处理其他逻辑？

5. 资源管控与成本熔断

怎么避免主 Agent 陷入逻辑死循环，疯狂创建子 Agent 导致**并发崩溃**和**成本爆炸**？

6. 信息隔离 (Context Isolation)

子 Agent 被唤起时，是否需要继承主 Agent 庞大的记忆？如何平衡**信息充分**与**上下文冗余污染**？